

GROUP 02 · CAPSTONE PROJECT

Auto Insurance AI Copilot

Accident Claims Triage · Repair Estimation · Policy Coverage · FNOL Bot

Domain	Auto / Motor Insurance
Format	End-to-End AI Copilot
Stack Hint	LangChain · FastAPI · FAISS · Streamlit · Docker · AWS
Duration	Day 19 & 20 — Full Capstone
Team Size	4–5 Participants
Type	Problem Statement Architecture Reference Evaluation Rubric

Enterprise AI Training Program · 15-Day Bootcamp Series

1. Project Overview

Group 02 is tasked with building a production-grade **Auto Insurance AI Copilot** — an intelligent assistant that serves policyholders, claims adjusters, and FNOL (First Notice of Loss) intake agents. The system must handle the full lifecycle of an auto insurance interaction: from policy coverage questions and accident triage, through repair cost estimation and claim submission guidance, to coverage dispute resolution.

This is a full end-to-end engineering capstone. Your team is responsible for all layers: data ingestion, vector indexing, LLM agent design, REST API, chat UI, safety guardrails, evaluation, containerization, and live deployment on AWS.

Domain	Auto / Motor Insurance
User Personas	Policyholders · Claims Adjusters · FNOL Intake Agents · Repair Shop Liaisons
Core Challenge	Multi-document RAG over policy and repair docs + structured data lookups (repair costs, coverage limits)
Final Deliverable	Live URL (AWS) + Streamlit UI + FastAPI + FAISS + Evaluation Scorecard
Duration	Day 19 & 20 (Full capstone + deployment demo)
Team Size	4–5 participants

2. Problem Statement

When a vehicle accident occurs, policyholders are under stress and need fast, accurate answers about what their policy covers, how to file a claim, whether their vehicle will be repaired or declared a total loss, and what their out-of-pocket exposure will be. Currently, this process involves long hold times, repetitive document checks by adjusters, and inconsistent guidance across channels.

Adjusters manually cross-reference policy documents against claim details, estimate repair costs using separate tools, and manage prior communications through disconnected systems. This creates delays, errors, and a poor experience at the most critical moment in the customer relationship — right after an accident.

■ Your Mission

Build an AI copilot that guides a policyholder or adjuster through the post-accident journey — answering natural-language questions about coverage, estimating repair costs, determining total-loss thresholds, guiding FNOL submission, and surfacing the relevant policy clause — all within 4 seconds, without hallucinating coverage or cost figures.

The system must distinguish between collision, comprehensive, liability, and uninsured motorist coverages, apply deductibles correctly, and refuse to guess when policy information is ambiguous or unavailable.

2.1 Key Pain Points Your System Must Address

- **Post-Accident Confusion:** Policyholders don't know which coverage type applies to their specific accident scenario (collision vs comprehensive vs liability).
- **FNOL Delays:** First Notice of Loss intake is slow — agents re-read policy documents to validate whether an incident qualifies for a claim.
- **Repair Cost Opacity:** Adjusters and policyholders have no quick way to estimate repair costs for common damage types against coverage limits.
- **Total Loss Determination:** The threshold for declaring a vehicle a total loss varies by state and policy — this information is buried in documents.

- **Rental Reimbursement Queries:** Policyholders are unsure whether rental car costs are covered during repair and for how long.
- **Coverage Dispute Handling:** When a claim is questioned, agents spend excessive time locating the exact policy clause that governs the decision.
- **Multi-Vehicle & At-Fault Scenarios:** Complex accidents involving multiple parties, partial fault, or uninsured drivers require nuanced policy interpretation.

■ ■ Non-Negotiable Safety Constraint

The copilot must NEVER provide a final claims decision or legal advice. Every response touching claim eligibility, fault determination, or coverage limits must cite its source document and page, and include a disclaimer directing the user to their assigned adjuster for official decisions. Fabricating repair cost estimates without data is a critical failure.

3. Provided Dataset & Knowledge Base

The following synthetic documents and structured datasets are provided as your knowledge base. All data is fictional and created for training purposes only. Your system must ingest, chunk, embed, and serve all sources through the vector store and structured lookups.

#	Document / Dataset	Format	Size	Primary Use Case
1	Auto Policy Declaration Pages (Standard / Premium / Comprehensive tiers)	PDF	8 pp × 3	Coverage types, limits, deductibles per tier
2	Full Auto Insurance Policy Document (Terms, conditions, exclusions)	PDF	72 pp	Detailed coverage terms, exclusions, definitions
3	FNOL Intake Guidelines & Claim Procedures	PDF	15 pp	Claim filing steps, documentation required, timelines
4	Repair Cost Reference Table (Common damage types × vehicle categories)	CSV	320 rows	Estimated repair cost ranges by damage + vehicle class
5	Total Loss Threshold Table (By state + vehicle age)	CSV	150 rows	State-specific total loss thresholds as % of ACV
6	Rental Reimbursement Schedule	PDF	4 pp	Daily limits, maximum days, covered scenarios
7	Uninsured / Underinsured Motorist Coverage Guide	PDF	10 pp	UM/UIM coverage rules, stacking, limits
8	Roadside Assistance & Towing Coverage Terms	PDF	5 pp	Towing limits, dispatch rules, covered scenarios
9	Glossary of Auto Insurance Terms	PDF	12 pp	Definitions: ACV, subrogation, deductible, betterment, etc.

■ Ingestion Hint

Pay careful attention to the two CSV datasets — repair cost and total loss thresholds are structured data, not free text. Consider how your agent should query these differently from the PDF policy documents. Metadata tagging by coverage_type (collision/comprehensive/liability/um) and policy_tier (standard/premium/comprehensive) will be critical for accurate retrieval.

4. Expected System Architecture & Data Flow

The diagram below defines the five layers your system must implement and the five numbered data flow steps your team must trace end-to-end. The architecture is identical in structure to all capstone groups — your differentiation is in the domain-specific tools, documents, and agent reasoning logic you build within this framework.

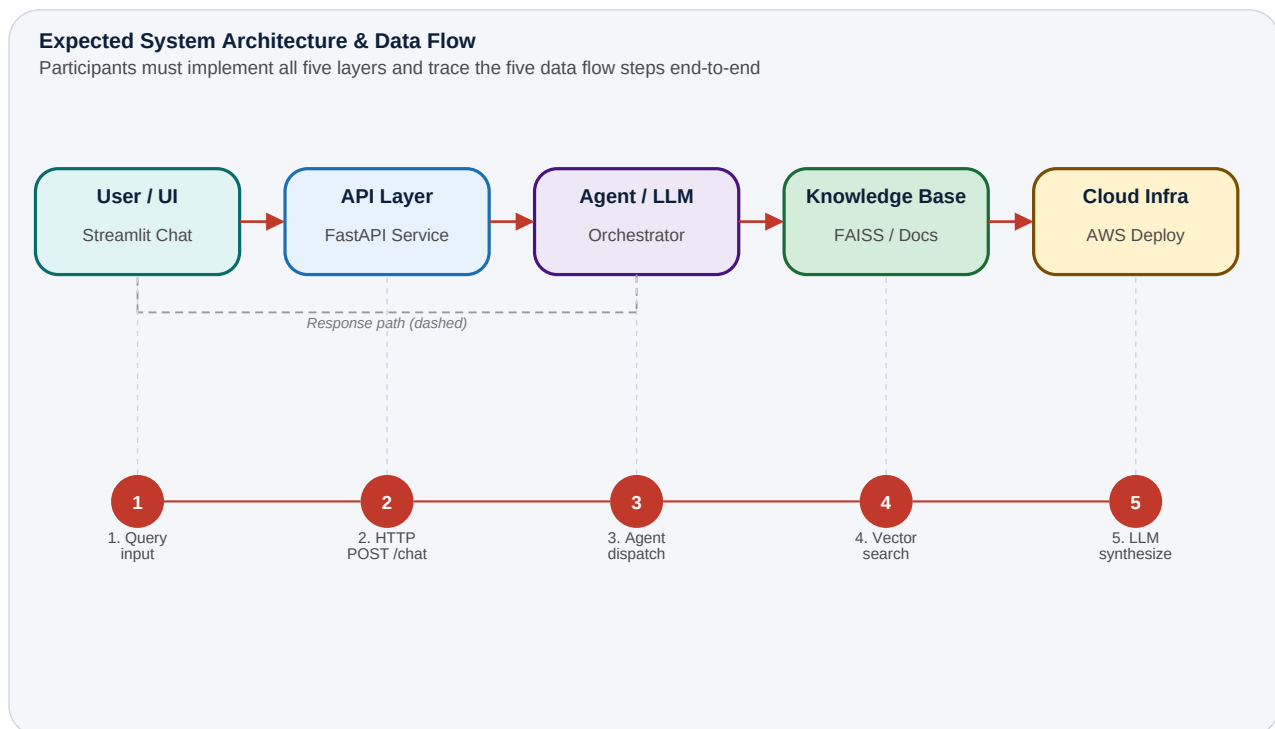


Figure 1: Five-layer reference architecture with end-to-end data flow steps

4.1 Layer Responsibilities

Layer	Component	Your Responsibility
User / UI	Chat Interface	A conversational front-end where policyholders or adjusters describe their accident scenario and ask questions. Must support session continuity, coverage type selection, and display of cited policy clauses.
API Layer	Backend Service	A REST API that validates incoming requests (query + session_id + coverage_type + policy_tier), routes them to the agent, and returns structured JSON with the answer and citations.
Agent / LLM	Orchestrator	An LLM-based agent that interprets the query, selects the correct tool (RAG, repair estimator, total loss checker, FNOL guide, rental lookup), applies guardrails, and synthesizes a grounded, cited response.
Knowledge Base	Vector Store + Docs	All provided PDFs must be chunked, embedded, and indexed in FAISS with metadata. The two CSVs (repair costs, total loss thresholds) require structured lookup handling — not just vector similarity.
Cloud Infra	Deployment Environment	The full stack must be Dockerized and deployed to AWS with a publicly accessible URL during the demo. Logs observable, secrets via env vars, no hardcoded credentials.

4.2 Required Data Flow (5 Steps)

Your implementation must demonstrate all five steps of the data flow during the live demo:

- **Step 1 — Query Input:** User describes an accident scenario or asks a policy question in the UI, optionally selecting their coverage type and policy tier.

- **Step 2 — HTTP POST /chat:** The UI sends the query, session ID, coverage type, and policy tier to the FastAPI endpoint. The API validates before forwarding.
- **Step 3 — Agent Dispatch:** The agent classifies the query intent (coverage Q&A, repair estimate, total loss check, FNOL guidance, rental lookup) and invokes the appropriate tool(s).
- **Step 4 — Vector Search / Structured Lookup:** For policy questions, the agent performs semantic search on FAISS. For cost or threshold queries, it performs structured CSV lookups filtered by vehicle category, state, or damage type.
- **Step 5 — LLM Synthesize:** The LLM generates a grounded response using retrieved context + memory, passes through guardrails, and returns a cited, safe answer with the source document reference.

5. Functional Requirements

Your system must fulfil the following capabilities. These are business requirements — implementation decisions are yours to make.

#	Requirement	Acceptance Condition
FR-01	Coverage Type Identification	User describes an accident → system correctly identifies which coverage type(s) apply (collision, comprehensive, liability, UM/UIM) and cites the policy section.
FR-02	Policy Q&A; with Citations	User asks any natural-language policy question → system returns an accurate answer citing source document, section, and page number.
FR-03	Repair Cost Estimation	Given vehicle category and damage type, system returns an estimated repair cost range from the reference table with applicable deductible applied.
FR-04	Total Loss Determination	Given vehicle ACV and estimated repair cost (and optionally state), system determines whether the vehicle meets the total loss threshold and explains the calculation.
FR-05	FNOL Guidance	User says they just had an accident → system walks them through the First Notice of Loss steps, required documentation, and filing deadlines per their policy.
FR-06	Rental Reimbursement Lookup	User asks about rental car coverage → system returns daily limit, maximum coverage days, and any restrictions for their policy tier.
FR-07	Policy Tier Awareness	System must distinguish between Standard, Premium, and Comprehensive policy tier responses. Coverage limits and deductibles differ — answers must reflect the correct tier.
FR-08	Conversation Memory	Within a session, prior context is retained (e.g., 'my Premium policy' stated earlier applies to subsequent turns without repetition).
FR-09	Source Citation	Every policy-grounded response must include: document name, section title, and page number.
FR-10	Guardrails	System must block: legal advice requests, fault determination requests, fabricated repair costs without data, PHI/PII in input, and prompt injection attempts.

5.1 Non-Functional Requirements

- **Performance:** API must respond within 4 seconds at the 95th percentile under normal load.
- **Reliability:** FAISS index and CSV lookup tables must persist across container restarts.
- **Security:** API keys, LLM credentials, and AWS credentials stored as environment variables — never hardcoded.
- **Observability:** All agent runs must be traceable — query, tool invoked, retrieved context, and final answer visible per turn.

- Portability:** Full application must run via a single Docker command with no local environment dependency beyond a .env file.

6. Evaluation Criteria & Scoring Rubric

Your capstone will be evaluated across five dimensions combining automated metric scoring (LangSmith + DeepEval) with a live demo assessment. **Total: 100 points.**

Dimension	Weight	What Will Be Assessed	Minimum Threshold
RAG Accuracy	25 pts	Faithfulness and Answer Relevancy on a 30-question test set covering all query types.	Faithfulness ≥ 0.85 Answer Relevancy ≥ 0.80
System Completeness	25 pts	All 10 functional requirements (FR-01 to FR-10) demonstrated live. Each = 2.5 pts.	8 of 10 FRs must pass
Guardrail Effectiveness	20 pts	4 adversarial inputs injected: legal advice request, fault determination, fabricated cost, prompt injection. Each blocked = 5 pts.	All 4 blocked
Deployment & Ops	20 pts	Live URL on AWS, Docker confirmed, observability traces shown, secrets via env vars.	Public URL live during demo
Presentation	10 pts	Architecture diagram presented, live system walkthrough, design decisions defended under Q&A.;	Must match implementation

6.1 Test Dataset Requirements (30 Q&A; Pairs)

Your team must prepare a 30-question test dataset before running the evaluation pipeline. Coverage must span all query categories:

Coverage Type Q&A; (RAG)	10 questions	Mix of collision, comprehensive, liability, UM/UIM scenarios
Repair Cost Estimation	6 questions	Different vehicle categories and damage types
Total Loss Determination	5 questions	Edge cases including borderline ACV scenarios
FNOL Guidance	5 questions	Different accident types and filing timelines
Rental & Ancillary Coverage	4 questions	Rental limits, towing, roadside assistance

Evaluation Tools

Use LangSmith (or equivalent) for run tracing and tool-call logging. Use DeepEval (or equivalent) to compute Faithfulness and Answer Relevancy on your 30-question set. Both scorecard outputs must be shown during the demo.

7. Deliverables & Demo Checklist

At the end of Day 20, each group will present a live demo. Items marked **CRITICAL** are required for a passing grade.

CRITICAL	Chat UI loads and accepts accident scenario queries with coverage type and policy tier selector
CRITICAL	Policy Q&A; returns answer with source document name, section, and page number
CRITICAL	Repair cost estimation returns a range from the reference table with deductible applied

CRITICAL	At least 4 of 10 functional requirements (FR-01 to FR-10) demonstrated live
CRITICAL	Application is live on a public AWS URL or EC2 IP during the demo window
REQUIRED	Total loss determination demonstrated with ACV and repair cost inputs
REQUIRED	FNOL guidance flow demonstrated end-to-end for at least one accident scenario
REQUIRED	LangSmith (or equivalent) traces shown with tool invocations for at least 5 queries
REQUIRED	DeepEval scorecard: Faithfulness and Answer Relevancy on 30-question test set
REQUIRED	Docker container confirmed: build and run demonstrated or evidenced
REQUIRED	Architecture diagram presented and matches implemented system
BONUS	Multi-party accident scenario handled (partial fault, uninsured driver) (+5 bonus pts)
BONUS	CI/CD pipeline or automated deployment script demonstrated (+3 bonus pts)

8. Suggested Timeline (Day 19 & 20)

The following time-boxes guide effort allocation. All phases must be complete before the demo window on Day 20.

Phase	Focus Area	Time Box	Exit Condition
Phase 1	Data ingestion — PDF loading, CSV parsing, FAISS indexing, metadata strategy	1.5 hrs	FAISS index built; repair cost and total loss CSVs queryable
Phase 2	Agent design — tool definitions, intent classification, prompt engineering, memory	2.0 hrs	Agent correctly handles at least 3 sample queries across different tools
Phase 3	API development — FastAPI endpoints, validation, error handling	1.5 hrs	FastAPI running on localhost, /chat returns valid JSON
Phase 4	UI development — Streamlit chat, session state, coverage selector, citation display	1.0 hrs	Streamlit UI connected to API and showing cited responses
Phase 5	Guardrails — adversarial testing, legal advice blocking, escalation logic	1.0 hrs	All 4 guardrail test categories blocked successfully
Phase 6	Evaluation — 30-question test set, DeepEval run, LangSmith trace review	1.0 hrs	Scorecard produced, Faithfulness $\geq$ 0.85
Phase 7	Docker build, AWS deploy, URL confirmed, demo rehearsal	1.0 hrs	Live URL accessible, demo rehearsed at least once

■ Final Reminder

This is a problem statement, not a solution guide. Your team must design the architecture, choose the implementation approach, and debug independently. Trainers will be available for unblocking critical issues only — not for step-by-step guidance.

Good luck, Group 02. Build something accurate, safe, and demo-ready.